

# Stage 2 — Project Requirements & Development Plan

## Stateful Application-Layer Firewall & Intrusion Filter

*Comprehensive System Programming & C++ Project Documentation*

**Author:** Ayush Behera

---

### Abstract

This document defines the functional and non-functional requirements, major system modules, core data structures, threat considerations, performance goals, expected deliverables, and development timeline for the Stateful Application-Layer Firewall & Intrusion Filter project. It serves as the project's official Project Requirements Document (PRD) and provides the technical foundation for moving from the initial project concept to the actual implementation. The project focuses on Linux system programming, modern C++ design, kernel-user space communication, stateful connection tracking, and application-layer packet inspection.

## 1. Executive Summary & Project Context

Modern networks face a wide range of attacks that cannot always be detected by traditional firewalls. Most basic firewalls operate mainly at Layers 3 and 4 of the OSI model, where they inspect information such as IP addresses, ports, and TCP flags. While this is useful for controlling network access, it does not provide much visibility into the actual application data being transmitted.

For example, malicious requests can be sent through otherwise legitimate HTTP or DNS connections. Since the traffic itself may use valid IP addresses and ports, a basic packet filter may allow it without inspecting what is contained inside the payload.

Another limitation of stateless filtering is the lack of connection context. Without keeping track of how a connection was established and how it is progressing, it becomes more difficult to identify invalid packets, unexpected state transitions, and certain types of flooding or spoofing attempts.

To address these limitations, the Stateful Application-Layer Firewall & Intrusion Filter, authored by Ayush Behera, is designed as a Linux-based network security daemon. The system combines packet interception through `libnetfilter_queue`, an in-memory stateful connection tracking table, and a modular Layer 7 inspection engine written in modern C++.

The overall goal is to create a lightweight system that can make security decisions using both connection state and application-level information.

## 2. Project Requirements Document (PRD)

### A. Functional Requirements

#### 1. Packet Interception Subsystem

- **Kernel-User Space Communication:** The system must communicate with the Linux networking stack through Netfilter and `libnetfilter_queue` to capture incoming, outgoing, and forwarded packets.
- **Explicit Packet Verdicts:** Every intercepted packet must be evaluated against the active security policies and assigned an appropriate verdict such as `NF_ACCEPT`, `NF_DROP`, or `NF_QUEUE` before being returned to the kernel.

- **Queue Management:** The interceptor should be able to handle bursts of network traffic while managing queue limits and packet information efficiently in user space.

## 2. Stateful Connection Tracking (conntrack)

- **5-Tuple Flow Identification:** The system must maintain an in-memory connection table where each network flow is identified using a 5-tuple: source IP address, destination IP address, source port, destination port, and transport protocol.
- **TCP State Tracking:** For TCP connections, the system must maintain important states such as `SYN_SENT`, `SYN_RECV`, `ESTABLISHED`, `FIN_WAIT`, `CLOSE_WAIT`, and `CLOSED`.
- **State Validation:** Packets that arrive in an unexpected order or belong to invalid or unestablished connections should be detected and handled according to the configured security policy.
- **Timeout and Cleanup:** The connection tracker must periodically remove stale and half-open connections. This prevents the table from continuously growing and reduces the possibility of memory exhaustion during connection floods.

## 3. Application-Layer (Layer 7) Inspection Engine

- **Protocol Payload Parsing:** The system must inspect unencrypted application-layer data. The initial implementation will focus on HTTP request methods such as `GET`, `POST`, `PUT`, and `DELETE`, along with URIs, request headers, User-Agent strings, and basic DNS queries.
- **Signature and Pattern Matching:** Extracted application data should be compared against known suspicious patterns, including basic SQL injection and Cross-Site Scripting (XSS) patterns, as well as other structural anomalies.
- **Extensible Parser Design:** The inspection engine should use a modular parser structure so that additional application protocols can be added later without significantly changing the existing code.

## 4. Configurable Rule & Signature Engine

- **Dynamic Rule Loading:** Security rules and signatures must be loadable, editable, and removable through configuration files or a runtime administrative interface.
- **Runtime Updates:** Changes to rules should be applied without restarting the firewall daemon or unnecessarily interrupting active network connections.
- **Threshold-Based Rate Limiting:** The rule engine should support configurable thresholds for detecting and limiting abnormal traffic rates, including potential brute-force attempts and traffic bursts.

## 5. Logging & Auditing Subsystem

- **Structured Event Logging:** The system must record important security events with timestamps, including dropped packets, invalid connection states, and detected signature matches.
- **Multiple Output Destinations:** Logs should be written to persistent files while also being available through the console for real-time monitoring and debugging.

## B. Non-Functional Requirements

## 1. Performance & Low Latency

- **Low Processing Overhead:** Packet inspection, connection-table lookups, and state processing should use as little CPU and memory as reasonably possible to avoid creating a network bottleneck.
- **Latency Target:** Under the defined test conditions, the system should target an average processing latency of less than *2ms* per packet.

## 2. Reliability, Concurrency & Memory Safety

- **Runtime Stability:** The daemon should operate reliably without memory leaks, buffer overflows, dangling pointers, or segmentation faults.
- **Modern C++ Resource Management:** The implementation should follow modern C++ practices, including smart pointers such as `std::unique_ptr` and `std::shared_ptr`, along with RAII (Resource Acquisition Is Initialization) for managing resources safely.
- **Thread Safety:** If multiple threads are used for packet processing or state management, shared data such as the connection table must be protected using appropriate synchronization mechanisms, thread-safe structures, or reader-writer locks.

## 3. Modularity, Portability & Extensibility

- **Separation of Responsibilities:** Packet capture, connection tracking, protocol parsing, and rule evaluation should be implemented as separate components with clearly defined interfaces.
- **Linux Compatibility:** The project should run on standard Linux distributions that provide Netfilter support and modern GCC or Clang toolchains.

### 3. System Architecture & Modular Breakdown

The system is divided into five major subsystems. Each subsystem has a specific responsibility, making the overall codebase easier to develop, test, and extend.

| Module                         | Core Responsibility                                                                         | Key Deliverables                                                                      |
|--------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>1. Interceptor Module</b>   | Communicates with the Netfilter queue, captures packets, and returns appropriate verdicts.  | <code>Interceptor.h/.cpp</code> wrapper around <code>libnetfilter_queue</code>        |
| <b>2. State Tracker</b>        | Maintains active connections and manages state transitions and timeouts.                    | <code>ConntrackTable.h/.cpp</code> , <code>Connection.h</code> state-management logic |
| <b>3. L7 Inspection Engine</b> | Extracts application-layer information and checks it against security rules and signatures. | <code>Inspector.h/.cpp</code> , rule parser, and pattern-matching components          |
| <b>4. Management CLI</b>       | Provides commands for loading rules and checking the firewall's current status.             | CLI executable ( <code>fw_admin</code> ) with command parser                          |
| <b>5. Logging Subsystem</b>    | Records security events and system activity in a structured format.                         | <code>Logger.h/.cpp</code> with file output and severity levels                       |

#### Detailed Module Specifications

##### Module 1: Packet Interceptor Subsystem (Interceptor)

The Interceptor acts as the main connection point between the Linux kernel and the user-space firewall application.

Packets can be redirected to a Netfilter queue using appropriate firewall rules. The Interceptor class initializes and manages the queue through `libnetfilter_queue` and registers a callback for incoming packets.

When a packet arrives, the callback extracts the required information, such as the packet ID, payload length, and raw packet data. This information is then converted into a suitable C++ representation and passed to the connection-tracking and inspection components.

After the packet has been processed, the interceptor sends the appropriate verdict back to the kernel, such as accepting or dropping the packet.

##### Module 2: Stateful Connection Tracker (ConntrackTable)

The Connection Tracker maintains an in-memory hash table containing currently active network flows.

Each connection is identified using a normalized 5-tuple. The corresponding connection object stores information such as the current TCP state, packet count, byte count, and the time when the connection was last seen.

For TCP traffic, the state machine checks important flags such as SYN, ACK, FIN, and RST against the current connection state. If a packet does not match an expected state transition, it can be marked as suspicious and handled according to the firewall rules.

A background cleanup routine will periodically remove connections that have exceeded their timeout period. This keeps the connection table from filling up with stale or half-open entries.

### **Module 3: Layer 7 Inspection Engine (Inspector)**

Once a connection has reached an appropriate state and contains application-layer data, the payload can be passed to the Layer 7 Inspector.

The inspector is responsible for identifying the application protocol and extracting relevant information. For HTTP traffic, this includes request methods, URIs, headers, and other available metadata.

The extracted content is then checked against configured security rules and signatures. For example, the engine may identify suspicious strings associated with SQL injection, XSS, command injection, or other application-layer attacks.

The inspection engine is designed to remain modular so that additional protocol parsers and detection rules can be added as the project develops.